O.N.E

Onboarding Navigation Environment

Autonomous Developer Onboarding Agent

Full Technical Plan — Free Stack Edition

Capstone Project I & II | 2025-26

Vivekanand Education Society's Institute of Technology

Department of Information Technology | Group No. 28

Team Members:
Archit Chitte | Parth Narkar | Harshvardhan Patil | Manas Patil
Guide: Dr. Rakhi Jadhav

Domain: AI | SDG 8 & 9

Executive Summary

O.N.E (Onboarding Navigation Environment) is an Autonomous Developer Onboarding Agent built with Agentic AI and Retrieval-Augmented Generation (RAG). The system guides new software developers through the complete onboarding process via a chat-based interface, personalising the experience based on role, experience level, and technology stack.

This document is the complete technical plan for both semesters of the capstone project. The entire system is built using a 100% free and open-source technology stack — there are no paid APIs, no paid cloud services, and no subscription tools required at any stage.

Two-Semester Implementation Strategy

	Semester 1 — Capstone I	Semester 2 — Capstone II
Focus	Core MVP — Functional AI Onboarding Platform	Advanced Enterprise Features + Verification
Goal	Working chatbot with RAG, checklist, and HR email	Multi-agent architecture, integrations, analytics
Deliverable	Functional MVP with all core modules	Production-ready enterprise onboarding platform
Stack	Gemini + ChromaDB + FastAPI + React + PostgreSQL	Same stack + advanced agents + real integrations

Technology Stack — Free Alternatives

Every paid service from the original plan has been replaced with a free equivalent:

Category	Original (Paid)	Free Alternative	Why
LLM / AI	OpenAI GPT-4 (paid API)	Gemini 2.0 Flash	Free tier: 15 req/min, 1500/day — enough for capstone
Embeddings	OpenAI text-embedding-3-small	Gemini text-embedding-004	Same free API key, 768-dim vectors, zero cost
Vector DB	Pinecone (paid cloud)	ChromaDB (local)	Runs in-process locally, no server, no account
Primary DB	Supabase (limited free)	PostgreSQL (local)	Full open-source DB, runs on any machine
Session Cache	Redis (paid managed)	In-memory dict / SQLite	Sufficient for sem 1 demo scope
Email (HR)	SendGrid / AWS SES	Gmail SMTP (free)	smtplib + Gmail App Password, zero cost
AI Framework	LangChain (heavy)	LangChain (free)	Keep — it is entirely free and open-source

Frontend	Next.js (complex)	React + Vite	Simpler, faster setup, same capabilities
Backend	FastAPI	FastAPI	Keep — it is entirely free and open-source
Hosting	AWS / GCP (paid cloud)	localhost	Run locally for sem 1 demo; Render.com free tier for sem 2

1. System Architecture — Semester 1

The Semester 1 architecture focuses on the five core modules needed for a working MVP: the chat interface, the agentic AI engine, RAG-based knowledge retrieval, checklist tracking, and HR notification. All components run locally with no cloud dependencies.

1.1 Architecture Overview

Layer	Technology	Rationale
Frontend (Chat UI)	React + Vite + TailwindCSS	Lightweight, fast dev server, component-based
Backend API	Python (FastAPI)	Async support, ideal for LLM + RAG ecosystem
Agent / LLM	Google Gemini 2.0 Flash	Free tier, function calling support, fast responses
AI Framework	LangChain	Orchestrates RAG pipeline and agent tool use, 100% free
Embeddings	Gemini text-embedding-004	Free with Google AI Studio key, 768-dim vectors
Vector Database	ChromaDB (local)	In-process, no server, no account, stores embeddings locally
Primary Database	PostgreSQL (local)	Relational DB for sessions, checklists, logs
Session Store	In-memory + DB fallback	Python dict for active sessions, persisted to PostgreSQL
Email Service	Gmail SMTP (smtplib)	Free with App Password, sends HR completion emails
Hosting	localhost (development)	FastAPI on :8000, React on :5173

1.2 Agent State Machine

The onboarding flow is modelled as a Finite State Machine (FSM). Each state has a specific purpose, and the agent transitions between states based on user actions and checklist progress.

State	Description	Trigger to Next State
WELCOME	Greet new hire, collect name, email, start date	Name and email collected
PROFILING	Ask conversational questions to determine role, experience level, tech stack, team	All profile fields extracted by LLM
PLAN_GENERATION	Generate personalised onboarding checklist based on developer persona	Checklist generated and saved to DB
ONBOARDING_EXECUTION	Walk through each checklist item step-by-step, answer questions, provide docs from RAG	All required items completed or user requests review
CHECKLIST_REVIEW	Show completion summary, allow marking items done or pending	User confirms ready to complete
COMPLETED	Trigger HR email, mark session complete in DB	Email sent successfully
FREE_QA	Available at any stage — user can ask questions answered via RAG	Triggered by question intent, returns to previous state

1.3 Data Flow

The end-to-end data flow for a new developer onboarding session:

1. HR registers employee in the system (POST /api/v1/auth/register-employee)
2. Employee receives an invite link and opens the chat interface in the browser
3. Employee sends first message — agent enters WELCOME state, collects name and email
4. Agent enters PROFILING state — extracts role, experience, stack via LLM from natural language
5. Agent enters PLAN_GENERATION — builds personalised checklist from master template filtered by persona
6. Agent walks through each checklist item — retrieves relevant docs from ChromaDB via RAG
7. User confirms each task — agent marks item complete in PostgreSQL via function call
8. On completion — agent confirms with user, calls send_hr_completion_email tool
9. FastAPI sends email via Gmail SMTP — HR receives structured report
10. Session marked as completed in PostgreSQL

2. Persona Detection Engine

The persona engine identifies three key dimensions about the new hire through natural conversational questions — without using a static form. This profile drives every personalisation decision downstream: which checklist items are included, how detailed the explanations are, which knowledge base documents are retrieved, and how the agent communicates.

2.1 Developer Persona Model

The persona captures the following structured fields:

Field	Type	Example Values
name	string	Jane Doe
email	string	jane.doe@company.com
start_date	date	2026-03-01
role	enum	backend, frontend, devops, fullstack, data
experience_level	enum	intern, junior, mid, senior
tech_stack	list of strings	python, django, postgres
team	optional string	Platform Engineering

2.2 Conversational Profiling Flow

The agent asks five conversational questions and uses Gemini to extract structured data from the natural language responses:

11. What is your name and email address?
12. What role are you joining as?
13. How would you describe your experience level?
14. What is your primary technology stack?
15. Which team are you joining?

Example extraction: the user responds 'I am a senior backend dev mostly working in Python and Go' — Gemini extracts: role=backend, experience_level=senior, tech_stack=[python, go]. This is done via a structured extraction prompt, not keyword matching.

3. Dynamic Onboarding Plan Generator

Based on the developer persona, the system generates a personalised onboarding checklist by applying three layers of filtering to a master template stored in the database.

3.1 Checklist Generation Logic

16. Load the master checklist template from PostgreSQL — this contains all possible onboarding items
17. Filter by role — DevOps gets CI/CD setup items; Frontend gets design system docs; Backend gets API guidelines
18. Filter by experience level — interns get detailed hand-holding steps; seniors skip basic setup items
19. Add tech-stack-specific items — Python developers get virtualenv setup; Node developers get npm/nvm setup

3.2 Checklist Item Data Model

Field	Type	Description
id	UUID	Unique identifier for the checklist item
title	string	Short title shown in UI (e.g. Setup Python environment)
description	text	Detailed description of what the developer needs to do
category	enum	access, tooling, documentation, compliance, team
required	boolean	Whether this item must be completed before HR notification
applicable_roles	list	Roles this item applies to (e.g. backend, fullstack)
applicable_levels	list	Experience levels this item applies to
applicable_stacks	list	Tech stacks this item applies to (e.g. python, node)
knowledge_base_refs	list	Document IDs in ChromaDB to retrieve for this item
status	enum	pending, in_progress, completed, skipped
completed_at	datetime	Timestamp when item was marked complete

3.3 Example Generated Checklist — Senior Backend Python Developer

#	Category	Item	Required

1	Access	GitHub org access and repo permissions	Yes
2	Access	Jira project board access	Yes
3	Access	Slack channels: #engineering, #backend, #incidents	Yes
4	Tooling	Local dev environment setup (Python 3.11, Poetry, Docker)	Yes
5	Tooling	IDE setup with linting and formatting configs	No
6	Documentation	Read Architecture Decision Records (ADRs)	Yes
7	Documentation	Review API design guidelines	Yes
8	Documentation	Review deployment runbook	Yes
9	Compliance	Sign NDA and IP agreement	Yes
10	Compliance	Complete security awareness training	Yes
11	Team	Schedule 1:1 with engineering manager	Yes
12	Team	Schedule buddy pairing session	Yes

An intern in the same role would additionally receive items such as: Read intro to Git workflow, Complete Python coding standards walkthrough, and more guided step-by-step setup instructions.

4. RAG Knowledge Retrieval System

The RAG (Retrieval-Augmented Generation) system allows the agent to answer developer questions using the company's own internal documentation. All documents are stored as vector embeddings in ChromaDB running locally — no cloud vector database is required.

4.1 Knowledge Base Structure

The company knowledge base is organised as a structured folder of markdown files:

- company/ — mission-and-values.md, org-structure.md, communication-guidelines.md
- engineering/ — architecture-overview.md, coding-standards.md, api-design-guidelines.md, git-workflow.md, code-review-process.md, adrs/
- setup/ — local-dev-python.md, local-dev-node.md, docker-setup.md, ide-configuration.md
- devops/ — ci-cd-pipeline.md, deployment-runbook.md, monitoring-alerting.md
- compliance/ — security-training.md, data-handling-policy.md, nda-ip-agreement.md
- team/ — backend-team-intro.md, frontend-team-intro.md, devops-team-intro.md

4.2 Document Ingestion Pipeline

Before the system can answer questions, all documents must be processed and stored in ChromaDB. The ingestion pipeline runs once at setup and again whenever documents are updated:

20. Load documents — LangChain document loaders handle Markdown, PDF, and plain text
21. Chunking — recursive text splitter creates 512-token chunks with 50-token overlap, preserving section headers as metadata
22. Embedding generation — Gemini text-embedding-004 converts each chunk to a 768-dimensional vector (free API)
23. Vector storage — embeddings and metadata stored in local ChromaDB collection on disk

Metadata attached to each chunk:

Metadata Field	Example Value	Purpose
source	setup/local-dev-python.md	Track document origin for source citations
category	setup	Enable category-based filtering
applicable_roles	backend, fullstack, data	Filter results by developer role
applicable_stacks	python	Filter results by tech stack
section_title	Installing Poetry	Help agent reference specific sections

doc_title	Local Development Setup - Python	Display friendly name in agent responses

4.3 Retrieval Strategy

The retrieval pipeline uses three steps to find the most relevant documents for each user query:

24. Semantic search — vector similarity search on the user query against all embeddings in ChromaDB
25. Metadata filtering — pre-filter by the developer's role and tech stack before ranking, ensuring only relevant documents are surfaced
26. Re-ranking — the top 10 results are scored by the LLM for contextual relevance, and the top 3 are passed to the agent prompt

4.4 Anti-Hallucination Guardrails

The agent is explicitly instructed to stay grounded in the knowledge base:

- Grounded generation — the system prompt states: Only answer based on the provided context. If the context does not contain the answer, direct the user to their manager or the engineering Slack channel.
- Source citation — every RAG-grounded response includes a [Source: filename] reference so the developer knows where the information came from
- Confidence threshold — if retrieval similarity scores are below 0.65, the agent adds a disclaimer that it is not fully confident in the answer
- Fallback escalation — after two failed retrieval attempts, the agent suggests asking the onboarding buddy or manager directly

4.5 LLM Prompt Architecture

Each message to the LLM includes four layers of context, assembled dynamically by the backend:

Prompt Layer	Content
System Prompt (per state)	Agent role, current FSM state, persona summary, current checklist item, instructions for this step
Retrieved Context (RAG)	Top 3 relevant documents retrieved from ChromaDB for this query or checklist item
Conversation History	Last 20 messages from the session (windowed to manage token limits)
User Message	The developer's latest message

5. Conversation Manager

The conversation manager maintains the full state of each onboarding session — including the FSM state, persona, checklist progress, and recent conversation history. In Semester 1, session state is stored in-memory for active sessions and persisted to PostgreSQL on state transitions.

5.1 Session State Model

Field	Type	Description
session_id	UUID	Unique identifier for this onboarding session
user_id	UUID	Reference to the developer's user record
current_state	FSMState	Current state in the finite state machine
persona	DeveloperPersona	The extracted developer profile (role, level, stack, team)
checklist	List[ChecklistItem]	The personalised checklist generated for this developer
current_checklist_index	int	Index of the current checklist item being worked on
conversation_history	List[Message]	Last 20 messages (role + content) for LLM context
created_at	datetime	Session start timestamp
last_active	datetime	Last message timestamp — used for idle detection

5.2 Message Handling Flow

For every incoming message from the developer, the backend executes the following steps:

27. Load session state from in-memory store (falls back to PostgreSQL if not in memory)
28. Classify message intent — question vs. task completion confirmation vs. general conversation
29. If question intent detected — run RAG retrieval to find relevant knowledge base documents
30. Build the full LLM prompt: system prompt for current state + retrieved docs + conversation history + user message
31. Call Gemini 2.0 Flash with the assembled prompt and the agent tool definitions
32. If the LLM returns a function/tool call — execute the tool (mark checklist item, query KB, send email)
33. Continue LLM conversation with tool results until no more tool calls are returned
34. Append user message and assistant response to conversation history

35. Persist updated state to PostgreSQL asynchronously
36. Return the assistant's response text to the frontend

6. Agent Tool / Function Calling Schema

The Gemini agent has access to a defined set of tools it can call during the conversation. Function calling allows the agent to take real actions — not just generate text — based on the developer's messages.

6.1 Available Tools

Tool Name	Description	Key Parameters
search_knowledge_base	Search company documentation for relevant information	query (string), category (optional: setup/engineering/compliance/team)
mark_checklist_item	Update the status of a checklist item	item_id (string), status (completed/skipped/in_progress), notes (optional string)
get_checklist_status	Retrieve the full current checklist with all statuses	none
send_hr_completion_email	Send the onboarding completion report to HR — only called when all required items are done	include_pending (boolean)
escalate_to_human	Escalate to a human buddy or manager when the agent cannot help	reason (string), channel (email)

6.2 Agent Tool Call Decision Rules

The agent uses these conversational cues to decide when to call each tool:

- User says 'Done with that' or 'I finished it' → call mark_checklist_item with status=completed
- User says 'I already have GitHub access' → call mark_checklist_item with status=completed and a note
- User says 'Can I skip this?' on a non-required item → call mark_checklist_item with status=skipped
- User asks a documentation question → call search_knowledge_base before responding
- User asks 'How am I doing?' or 'What is left?' → call get_checklist_status
- Agent cannot answer a question after 2 retrieval attempts → call escalate_to_human
- All required items completed and user confirms → call send_hr_completion_email

7. Database Design (PostgreSQL)

PostgreSQL is used as the primary relational database. It stores all onboarding sessions, checklist items, users, and conversation logs. The database runs locally and requires no paid cloud service.

7.1 Core Tables

Table	Purpose	Key Columns
users	Developer and HR admin accounts	id (UUID), name, email, role (employee/hr_admin), created_at
onboarding_sessions	One record per onboarding run	id, user_id (FK), persona (JSONB), status (in_progress/completed/abandoned), started_at, completed_at, hr_notified
checklist_items	Individual tasks per session	id, session_id (FK), item_key, title, description, category, required, sort_order, status (pending/in_progress/completed/skipped), completed_at, notes
conversation_logs	Full message history per session	id, session_id (FK), role (user/assistant/system), content (text), metadata (JSONB), created_at
knowledge_documents	Metadata for ingested KB documents	id, source_path, category, applicable_roles, applicable_stacks, last_ingested_at

7.2 Checklist Item Status Transitions

Transition	Trigger	Who Sets It
pending → in_progress	Agent begins walking through the item	Agent (tool call)
in_progress → completed	User confirms completion	Agent (mark_checklist_item tool)
in_progress → skipped	User requests skip on a non-required item	Agent (mark_checklist_item tool)
completed → in_progress	User says they need to redo the task	Agent (mark_checklist_item tool)

8. Backend API Design (FastAPI)

The FastAPI backend exposes a clean REST API consumed by the React frontend. All routes are async and use SQLAlchemy for database operations.

8.1 Core API Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Employee or HR admin login — returns JWT token
POST	/api/v1/auth/register-employee	HR creates a new employee account for onboarding
POST	/api/v1/onboarding/start	Start a new onboarding session — creates session in DB
GET	/api/v1/onboarding/{id}	Get full session details including persona and status
GET	/api/v1/onboarding/{id}/progress	Get checklist progress with counts and percentage
POST	/api/v1/chat/{session_id}/message	Send a message and receive the agent response
GET	/api/v1/chat/{session_id}/history	Retrieve full conversation history for a session
PATCH	/api/v1/checklist/{item_id}	Manually update a checklist item status
GET	/api/v1/checklist/{session_id}	Get the full personalised checklist for a session
POST	/api/v1/knowledge/ingest	Admin endpoint to ingest new documents into ChromaDB
POST	/api/v1/knowledge/query	Direct RAG query endpoint for testing retrieval quality
GET	/api/v1/admin/sessions	List all onboarding sessions (paginated) for HR dashboard
GET	/api/v1/admin/metrics	Aggregate metrics: completion rate, average duration, etc.

8.2 Progress API Response Format

The GET /progress endpoint returns a structured JSON response used by the frontend to render the checklist sidebar and progress bar:

Field	Type	Description
session_id	string	Unique session identifier

user	string	Employee name
status	string	in_progress or completed
total_items	integer	Total number of checklist items
completed	integer	Number of completed items
pending	integer	Number of pending items
skipped	integer	Number of skipped items
percent_complete	float	Completion percentage (0-100)
current_item	object	Title, category, and status of the active item
items	array	Full list of all checklist items with statuses

9. Chat Interface — Frontend (React + Vite)

The frontend is a React single-page application built with Vite and styled with TailwindCSS. It provides the chat interface for developers and a separate HR admin dashboard.

9.1 Developer Chat Interface Features

- Chat window with message bubbles — user messages on the right (blue), agent messages on the left (gray)
- Typing indicator — animated dots shown while the agent is processing the message
- Checklist sidebar — real-time checklist showing all items with their current status (pending / in progress / done / skipped)
- Progress bar at the top — shows overall completion percentage, updates as items are marked complete
- Quick action buttons — Mark as done, Skip, Ask a question — reduce typing for common actions
- Document preview panel — when the agent references a knowledge base document, an expandable panel shows the relevant section
- Message history — scrollable full conversation history persists within the session

9.2 HR Admin Dashboard Features

- Session list — table of all active and completed onboarding sessions with name, role, status, and start date
- Per-employee progress view — click any session to see the full checklist breakdown
- Completion metrics — average onboarding time, completion rate by role, sessions completed this week
- Filter controls — filter sessions by role, team, status, and date range
- Manual HR notification trigger — resend the completion email for any completed session

9.3 Tech Stack Details

Tool	Version	Purpose
React	18+	UI component library
Vite	5+	Dev server and build tool (replaces Next.js, much simpler)
TailwindCSS	3+	Utility-first styling — no CSS files needed
Axios	latest	HTTP client for FastAPI communication
React Router	v6	Client-side routing between chat and dashboard views

10. HR Notification Service

When all required checklist items are complete, the agent sends a structured completion report to HR via Gmail SMTP — entirely free using Python's built-in smtplib library.

10.1 Trigger Conditions

37. All required checklist items are marked as completed
38. OR the user explicitly requests completion with pending items flagged in the report
39. The agent confirms with the user before sending: 'You have completed 11 of 12 items. Shall I send your completion report to HR?'

10.2 Email Setup — Gmail SMTP (Free)

Configuration steps to send emails for free:

40. Create a dedicated Gmail account (e.g. one.onboarding.agent@gmail.com)
41. Enable 2-Factor Authentication on the Gmail account
42. Generate an App Password from Google Account settings
43. Set GMAIL_ADDRESS and GMAIL_APP_PASSWORD as environment variables in the backend
44. Use Python smtplib with SMTP_SSL to send via smtp.gmail.com on port 465

10.3 Email Template — Completion Report

Section	Content
Subject	Onboarding Completed — [Name] ([Role])
Employee Details	Name, email, role, team, tech stack, start date
Completed Items	Numbered list of all completed checklist items with checkmarks
Pending / Skipped Items	List of any skipped or still-pending items
Completion Details	Timestamp, total duration, agent session ID
Footer	Automated notification from O.N.E system

10.4 Confidence Score Calculation

The HR report includes a completion confidence score calculated as: (number of required items completed) / (total required items) * 100. In Semester 1 this is self-reported confidence since task verification is a Semester 2 feature.

11. Semester 1 Sprint Plan

Week	Sprint Focus	Deliverables
Week 1	Foundation and Setup	PostgreSQL schema design, project scaffolding, Docker setup (optional), FastAPI project structure, knowledge base ingestion pipeline, ChromaDB local setup
Week 2	Core AI Engine	Gemini 2.0 Flash integration, FSM state machine implementation, persona profiling agent, checklist generation from persona, basic chat loop working end-to-end
Week 3	RAG + Intelligence	ChromaDB vector store setup, Gemini embedding generation, document chunking and ingestion, retrieval pipeline, prompt engineering, source citation, anti-hallucination guardrails
Week 4	Chat UI + Checklist	React + Vite chat interface, checklist sidebar, progress bar, real-time status updates via polling or SSE streaming
Week 5	HR Notification + Dashboard	Gmail SMTP email service, completion report template, HR admin dashboard, session list and metrics view
Week 6	Testing and Polish	End-to-end testing of full onboarding flow, LLM evaluation with golden test cases, bug fixes, documentation, demo preparation

11.1 Semester 1 Final Deliverables

- AI onboarding chatbot — functional end-to-end chat with Gemini agent
- Persona detection — role, experience, and stack extracted from natural conversation
- Personalized onboarding workflow — dynamic checklist generated per developer
- RAG documentation retrieval — answers grounded in company knowledge base via ChromaDB
- Checklist tracking — real-time progress with status updates in the UI
- HR notification system — automated completion email via Gmail SMTP
- HR admin dashboard — session list, progress view, and basic metrics

12. Testing Strategy — Semester 1

Test Type	Scope	Tools
Unit Tests	Persona extraction, checklist generation logic, RAG retrieval functions, email formatting	pytest
Integration Tests	FastAPI endpoints, PostgreSQL operations, ChromaDB retrieval pipeline	pytest + httpx
E2E Tests	Full onboarding flow from first message to HR email	Manual testing for sem 1; Playwright in sem 2
LLM Evaluation	Response quality, hallucination detection, correct tool call selection	Custom golden dataset evaluation framework
RAG Evaluation	Retrieval precision — does the right document come back for each question?	Precision@K measurement on test question set

12.1 LLM Golden Test Cases

A set of representative test cases is maintained to evaluate the agent's behaviour on known inputs:

Input	Expected Behaviour
What version of Python do we use?	Retrieves local-dev-python.md, responds with 3.11, cites source
I have finished setting up my local environment	Calls mark_checklist_item with status=completed
Can I skip the IDE setup?	Checks if item is required, if not calls mark_checklist_item with status=skipped
What is the best pizza place near the office?	Graceful redirect — says this is outside the onboarding knowledge base
I am done with everything	Calls get_checklist_status, checks all required items, offers to send HR email

13. Semester 2 Overview

Semester 2 transforms the working MVP into a production-quality enterprise onboarding platform. The primary additions are the Task Verification Engine (solving the trust problem from Semester 1), a multi-agent architecture, real external integrations, a full analytics dashboard, and an extended onboarding lifecycle beyond Day 1.

The free technology stack from Semester 1 is retained throughout Semester 2. No paid services are introduced.

13.1 Semester 2 Architecture Upgrades

Component	Semester 1	Semester 2 Upgrade
LLM Architecture	Single Gemini call per message	Multi-agent: Router + Conversation + Verification + Summarization agents
Session State	In-memory dict + PostgreSQL	Redis (free, local) for fast session access during active sessions
Task Completion	Self-reported — user says done	6-tier verification engine with API checks, quizzes, and proof submission
Integrations	Mock / simulated	Real GitHub API, Gmail SMTP, Google Calendar API (all free)
Analytics	Basic session list in HR dashboard	Full bottleneck analysis, time-per-task metrics, knowledge gap tracking
Onboarding Lifecycle	Ends at checklist completion	30-60-90 day plan with milestone check-ins
Knowledge Base	Static folder of markdown files	Admin management portal, versioning, staleness detection, gap analysis

14. Task Verification Engine

The core upgrade in Semester 2. The Semester 1 system trusts the user's self-report — saying 'done' to every task means a developer can complete onboarding in 90 seconds without doing any actual work. Semester 2 moves to a verify-then-trust model where every task type has an appropriate verification mechanism.

14.1 Verification Tier Taxonomy

Six verification tiers are defined, each appropriate for a different type of task:

Tier 1 — System-Verified (Fully Automated)

Tasks that can be verified by calling external APIs. The user never self-reports — the system checks directly.

Task	Verification Method
GitHub org access	GitHub API: check if user is a member of the org and has correct repo permissions
Docker installed	User runs docker --version, pastes output — agent validates version number meets minimum
Local dev server running	User runs a health check endpoint and pastes the JSON response — agent validates expected fields

Tier 2 — Knowledge Verification (Comprehension Quiz)

For documentation tasks — 'read the architecture doc', 'review coding standards' — the agent generates 3-5 quiz questions from the document content after the user claims completion. Questions are generated dynamically by the LLM so they vary between hires. A minimum passing score of 60% is required to mark the task verified. The difficulty of questions scales with the developer's experience level.

Experience Level	Quiz Style
Intern	Basic recall — what does this document say about X?
Junior	Applied understanding — given this policy, what would you do in situation Y?
Mid	Scenario-based — describe the correct approach for Z based on our standards
Senior	Architectural judgment — evaluate this design decision against our ADRs

Tier 3 — Proof-of-Work Submission

For environment setup tasks, the developer submits actual command output — not screenshots which can be faked.

Task	Required Proof
Local dev environment setup	Paste output of curl localhost:8000/docs — agent validates Swagger UI HTML markers are present
Docker installed	Paste output of docker --version and docker-compose --version — agent validates version numbers
IDE configured	Run a lint command on a provided test file and paste the output — proves formatter is working
Git workflow	Create a test branch with the correct naming convention, make a conventional commit, push to the sandbox repo

Tier 4 — Peer Confirmation

For tasks that inherently require another human — scheduling a 1:1, attending standup, buddy pairing. When the developer claims completion, the system emails the relevant peer (manager, buddy, team lead) with a confirm/deny link. The task remains as 'awaiting confirmation' until the peer responds. This is non-blocking — the developer continues with other tasks while waiting.

Tier 5 — Timed Reading with Engagement Tracking

For compliance documents (NDA, data handling policy, security policy). The document is presented in a reading pane and the system tracks time spent, scroll depth, and interaction signals. The minimum reading time is calculated from document length at 250 words per minute. After meeting the time threshold, a comprehension quiz is triggered (Tier 2). If the developer scrolls through a 2000-word document in 30 seconds, the agent gently flags it and asks them to read more carefully.

Tier 6 — Self-Attestation (Last Resort)

Only for tasks where none of the other tiers apply. The developer explicitly confirms a statement (e.g. 'I confirm I have read and understood the Mission and Values document'), this is timestamped and logged in the HR report with a flag indicating it is self-reported.

14.2 Verification Confidence Scoring

Verification Method	Confidence Weight	HR Report Label
System-verified (API check)	100%	Full confidence
Quiz passed with score 80% or above	95%	High confidence
Proof submitted and validated	90%	High confidence
Peer confirmed	95%	High confidence

Timed reading plus quiz	85%	Medium-high confidence
Self-attestation only	50%	Low confidence — flagged for HR review

The overall verification confidence score in the HR email is a weighted average across all checklist items. If a significant portion are self-attested, the report is flagged for HR review.

15. Multi-Agent Architecture (Semester 2)

Instead of a single monolithic LLM call per message, Semester 2 splits the agent into four specialised sub-agents. All use Gemini models at different capability levels to balance speed and cost — all within the free tier.

15.1 Agent Roles

Agent	Model Tier	Responsibility
Router Agent	Gemini Flash (fast, cheap)	Classifies every message into intent: task_completion_claim, question, off_topic, frustration_signal, help_request. Runs in under 200ms.
Conversation Agent	Gemini Flash (main workhorse)	Handles the actual onboarding conversation, guides through tasks, presents information, manages FSM flow.
Verification Agent	Gemini Flash (complex reasoning)	Generates quiz questions from document content, validates proof submissions, interprets ambiguous completion claims.
Summarization Agent	Gemini Flash (lightweight)	Compresses older conversation history to maintain context across long sessions without exceeding token limits.

15.2 Adaptive Pacing Engine

The system tracks each developer's performance and dynamically adjusts the onboarding pace:

- Fast-track detection — senior engineer answers quiz questions correctly on first try and completes tasks quickly: agent condenses explanations and moves faster
- Struggle detection — developer fails a quiz twice or takes a long time on a task: agent slows down, offers more detail, suggests scheduling time with the buddy
- Sentiment analysis — Router Agent runs a lightweight sentiment check on each message; if frustration or confusion is detected, tone shifts to more encouraging and a break is suggested

15.3 Proactive Agent Behaviours

The agent does not only respond to messages — it proactively reaches out:

- Daily nudge — if the developer has not interacted in 24 hours, a reminder email is sent: You have 4 tasks remaining. Want to pick up where you left off?
- Deadline awareness — if the developer is 3 days into a 5-day onboarding window and only 40% complete, the agent flags the most critical remaining compliance tasks
- Manager notification — if a developer is stuck (failed a quiz 3 times, no activity for 48 hours), the engineering manager is notified so they can step in

- Dependency enforcement — if Task B requires Task A (e.g. local dev setup requires Docker), the agent enforces ordering and explains why

16. Knowledge Base Management System (Semester 2)

16.1 Admin Document Management Portal

A web admin panel for engineering leads and HR to manage the knowledge base without touching code:

- Document CRUD — upload, edit, tag, version, and retire documents through the UI. Supports Markdown, PDF, and plain text.
- Version control — every document edit creates a new version. The system tracks which version each hire was onboarded against.
- Staleness detection — documents have a last_reviewed_at field. Documents not reviewed in 90 days are flagged, get a lower retrieval confidence score, and show a warning to the developer.
- Gap analysis dashboard — the system logs every question that could not be answered from the knowledge base. The admin portal surfaces these as a ranked list of missing documentation topics.

16.2 RAG Pipeline Improvements

Improvement	What It Solves
Hierarchical chunking	Instead of fixed 512-token chunks, parse by document structure (headings, sections). Ensures coherent sections are retrieved rather than arbitrary text windows.
Hypothetical Document Embeddings (HyDE)	Before searching, the LLM generates a hypothetical ideal answer, which is then embedded and used as the search query. Dramatically improves retrieval for natural-language questions that use different terminology from the docs.
Cross-encoder re-ranking	After initial vector retrieval of top 20 candidates, a cross-encoder scores them for actual relevance. Catches cases where embedding similarity is high but semantic relevance is low.
Contradiction detection	If two retrieved documents contain conflicting information, the agent flags it: I found two different answers in our docs — please confirm with your team lead.

17. Real External Integrations (Semester 2)

All integrations in Semester 1 are mocked. In Semester 2 they are replaced with real API calls — using only free APIs and free tiers.

17.1 GitHub Integration (Free API)

- Verify org membership — GitHub API checks if the developer has been added to the org
- Verify repo permissions — checks if the developer has the correct access level on required repos
- Onboarding sandbox repo — a dedicated repository for new hire exercises (create branch, make commit, open PR)
- PR verification — system monitors the sandbox repo for PRs from the new hire and checks branch naming convention, commit format, and CI status

17.2 Google Calendar Integration (Free API)

- Auto-scheduling — when the checklist includes 'schedule 1:1 with manager', the system checks Google Calendar availability for both parties and suggests open time slots
- Onboarding timeline — generate a calendar overlay showing the recommended onboarding schedule day by day

17.3 Email Drip Campaign (Gmail SMTP)

For developers who do not engage with the chat interface, structured email reminders are sent:

Day	Email Content
Day 0	Welcome email with onboarding link
Day 1 AM	Reminder: Have you set up your accounts?
Day 2 AM	Prompt: Time to set up your dev environment
Day 3	Documentation review reminder
Day 5	Final push: Complete your compliance tasks to finish onboarding

17.4 n8n Workflow Automation (Free, Self-Hosted)

n8n is a free open-source workflow automation tool (like Zapier but self-hosted). In Semester 2 it can be used to orchestrate multi-step agent workflows without writing custom code — for example, triggering the onboarding agent when a new employee is created, sending Slack messages, or chaining API calls for access provisioning.

18. Analytics and Intelligence Dashboard (Semester 2)

18.1 Onboarding Analytics for Engineering Leadership

Metric Category	Metrics
Time-to-Productivity	Average time from start to completion by role, by level, by team. Trend over time — is onboarding getting faster?
Bottleneck Identification	Which tasks take longest? Which tasks have highest quiz failure rate? Where do developers ask the most questions?
Knowledge Base Health	Most-accessed documents. Documents never accessed. Search queries with low retrieval confidence. Staleness report.
Drop-off Analysis	At which checklist item do developers stop engaging? Correlation between drop-off and specific documents or tasks.

18.2 Manager Dashboard

- View all current and past hires on their team with per-item progress breakdown
- Alerts when a developer is stuck — no activity for 48 hours, repeated quiz failures
- Comparison view — how this hire is progressing versus the team average for their role and level
- Action items — tasks that are pending their confirmation (buddy calls, 1:1s)

18.3 HR Compliance Dashboard

- Compliance audit trail — downloadable report showing exactly when each compliance task was completed, how it was verified, and by whom
- Cohort analysis — how does the current batch of new hires compare to the previous quarter
- Post-onboarding survey results correlated with agent interaction data

19. Extended Onboarding Lifecycle (Semester 2)

Onboarding does not end when the checklist is complete. Semester 2 extends the system into a structured 30-60-90 day journey.

19.1 30-60-90 Day Plan

Milestone	Agent Action
Day 30 check-in	Agent re-engages with a short survey: How are you settling in? Surfaces relevant documentation they may not have seen during initial onboarding.
Day 60 milestone	Verifies the developer has shipped their first meaningful PR, attended key team rituals, and has a healthy code review cadence. Notifies the manager with a progress summary.
Day 90 review	Triggers the formal review process. Generates a comprehensive report for the manager covering onboarding performance, quiz scores, engagement metrics, and time-to-first-PR.

19.2 Re-Onboarding for Role Changes

When a developer switches teams or is promoted, the system generates a delta onboarding plan — only the tasks relevant to the new role that were not covered in the original onboarding. No redundant repetition of already-completed items.

19.3 Knowledge Refresh

Every quarter, developers are nudged to review documentation that has changed significantly since their onboarding. If the architecture migrated from RabbitMQ to Kafka, all developers who were onboarded before the migration receive a targeted refresh notification with a summary of what changed.

20. Security Design

20.1 Authentication and Authorization

- JWT tokens — all API routes are protected with JWT authentication
- RBAC — four roles: new_hire (view own progress, chat), onboarding_buddy (view assigned buddy, confirm tasks), engineering_manager (view team progress, view analytics), hr_admin (all sessions, compliance reports, knowledge base management)
- Password hashing — bcrypt for all stored passwords

20.2 PII Handling

- Employee names, emails, and personal details are treated as sensitive data
- Conversation logs are access-controlled — developers can only see their own logs
- CORS policies restrict which origins can call the FastAPI backend

20.3 Prompt Injection Prevention

- Input sanitisation — user messages are screened for known injection patterns before being passed to the LLM
- Output validation — agent responses are post-processed to ensure they do not contain credentials, internal URLs not in the knowledge base, or instructions to bypass onboarding steps
- Sandboxed tools — the agent can only call the predefined tool set with validated parameters

20.4 Audit Logging

Every significant action is logged to PostgreSQL with a timestamp: task completed, quiz attempted, proof submitted, verification passed or failed, HR email sent, document viewed, escalation triggered. This log is queryable and exportable for compliance reporting.

21. Semester 2 Sprint Plan

Phase	Weeks	Deliverables
Phase 1: Verification Engine	1-4	6-tier task verification, quiz generation from ChromaDB documents, proof-of-work submission and validation, peer confirmation via email, timed reading tracker, verification confidence scoring
Phase 2: Real Integrations	5-8	Live GitHub API integration (org membership, repo permissions), Google Calendar API for auto-scheduling, n8n workflow automation, email drip campaign for inactive users
Phase 3: Analytics and Admin	9-11	Full analytics dashboard with bottleneck metrics, admin knowledge base management portal, HR compliance audit trail, document versioning and staleness detection, gap analysis dashboard
Phase 4: Scale and Polish	12-14	Multi-agent architecture (Router + Verification agents), adaptive pacing engine, RAG improvements (HyDE + cross-encoder re-ranking), performance optimisation, security hardening, accessibility audit
Phase 5: Extended Lifecycle	15-18	30-60-90 day milestone plans, re-onboarding for role changes, quarterly knowledge refresh notifications, post-onboarding survey, feedback loop into knowledge base improvements

22. Risks and Mitigations

Risk	Impact	Mitigation
Gemini free tier rate limit hit	Slow responses during demo with multiple concurrent users	Add retry logic with exponential backoff; cache frequent query responses; switch to longer session polling
LLM hallucination	Incorrect onboarding information delivered to developer	RAG grounding, confidence scoring, source citations, explicit guardrail prompt instructions
ChromaDB data loss on machine wipe	Vector index must be rebuilt from scratch	Persist ChromaDB to a committed directory; document rebuild script; keep source markdown files in Git
Prompt injection	Developer manipulates agent to skip compliance tasks	Input sanitisation, system prompt hardening, output validation post-processing
Knowledge base staleness	Outdated information served to developers	Document last_reviewed_at tracking, staleness warnings in UI, admin email nudges
Scope creep	Delayed delivery	Strict sem 1 scope of 5 core modules only; advanced features locked to sem 2; mocked integrations in sem 1

23. Future Enhancements (Post-Capstone)

- Slack / Microsoft Teams bot — full onboarding via chat platform DMs without opening the web UI
- Live access provisioning — automatically create GitHub accounts, Jira projects, Slack channels on hire creation
- Multi-language support — agent communicates in the developer's preferred language
- Voice interface — speech-to-text onboarding for accessibility use cases
- Multi-tenant SaaS — white-label the platform for multiple organisations with isolated knowledge bases
- Mobile app — React Native companion app with push notifications for reminders and peer confirmations
- LMS integration — auto-enrol new hires in compliance training courses, pull certificate completion as verification proof

24. References

45. A Multi-agent Onboarding Assistant based on Large Language Models, Retrieval Augmented Generation, and Chain-of-Thought. arXiv:2503.23421, 2025.
46. AI-driven chatbot as a support tool for developers during the onboarding process. Theseus, 2023.
47. Revolutionizing Newcomers' Onboarding Process in OSS Communities: The Future AI Mentor. arXiv:2505.04277, 2024.
48. Large Language Model-Based Agents for Software Engineering. arXiv:2409.02977, 2024.
49. From LLMs to LLM-based Agents for Software Engineering: A Survey. arXiv:2408.02479, 2024.
50. LangChain Documentation. https://docs.langchain.com
51. ChromaDB Documentation. https://docs.trychroma.com
52. Google Gemini API Documentation. https://ai.google.dev
53. FastAPI Documentation. https://fastapi.tiangolo.com